



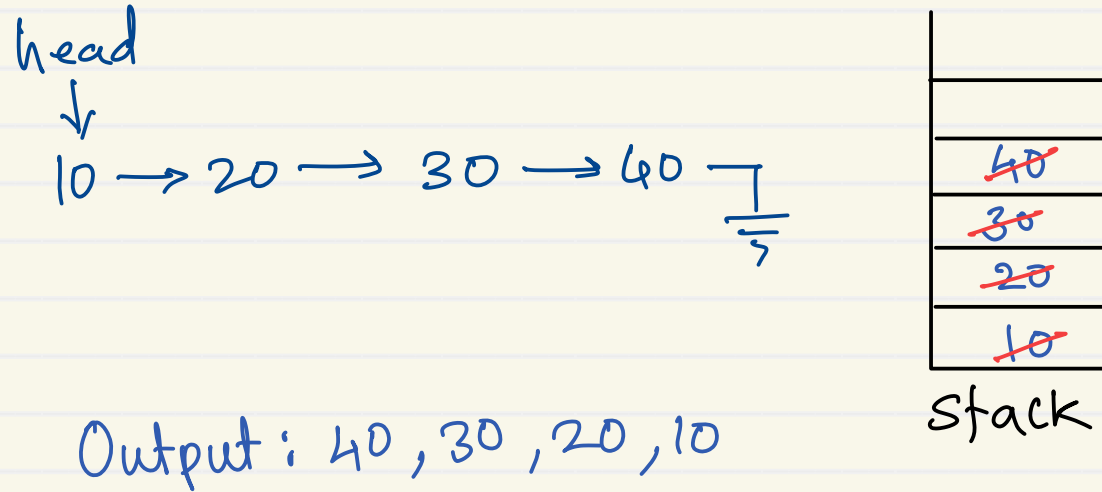
Sunbeam Institute of Information Technology
Pune and Karad

Data structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

- Display singly linked list in reverse order. (using stack)



Algorithm:

- push all node values on stack
- while stack is not empty, pop value from stack and display it.

```
Stack<Integer> st = new Stack<>();
```

```
Node trav = head;
while (trav != null) { ← n itrs
    st.push(trav.data);
    trav = trav.next;
}
```

```
while (!st.isEmpty()) { ← n itrs
    val = st.pop();
    sysout(val);
}
```

Time:

- to push : n itrs
- to pop : n itrs
- total itrs = $2n$
- $T \propto n$

$$T(n) = O(n)$$

Space:

Aux space : stack

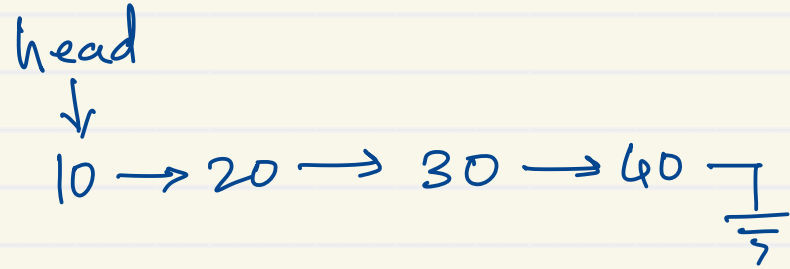
$S \propto n$

$$S(n) = O(n)$$

Linked List - competitive/interview question

- Display singly linked list in reverse order.

(using recursion)



```
void rDisplay(Node trav) {
    if (trav == null)
        return;
    rDisplay(trav.next);
    sysout(trav.data);
}
```

Time :

No. of recursive calls = $n+1$

$T \propto n+1$

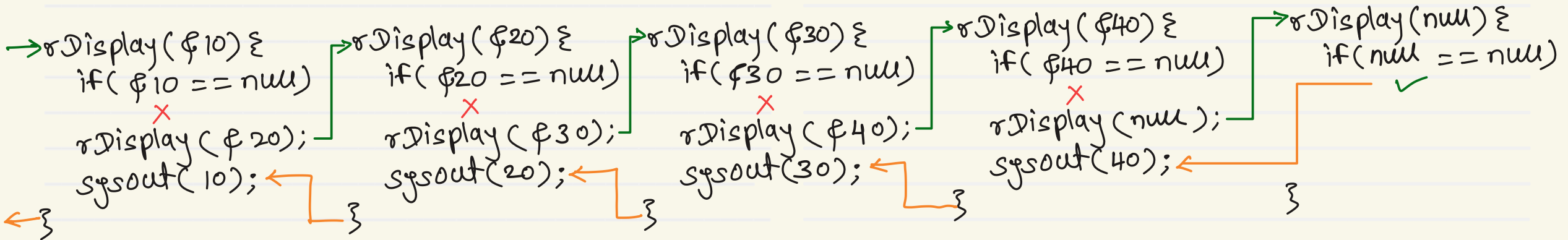
$$T(n) = O(n)$$

Space :

Aux space $\propto n$

$$S(n) = O(n)$$

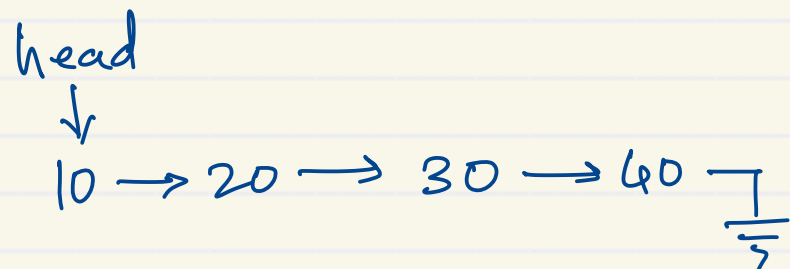
Output : 40, 30, 20, 10



Linked List - competitive/interview question

- Display singly linked list in forward order.

(using recursion)



Output : 10, 20, 30, 40

```
void fDisplay(Node trav) {  
    if (trav == null)  
        return;  
    sysout(trav.data);  
    fDisplay(trav.next);  
}
```

Time :

No. of recursive calls = $n+1$

$T \propto n+1$

$$T(n) = O(n)$$

Space :

Aux space $\propto n$

$$S(n) = O(n)$$

fDisplay(null)
fDisplay(\$40)
fDisplay(\$30)
fDisplay(\$20)
fDisplay(\$10)

Stack

- function calling itself directly is called as direct recursion

e.g. func() {

func()

}

- there are two types of direct recursion

Tail Recursion

- recursive call is last statement of recursive function

```
mid fDisplay(Node trav) {  
    if (trav == null)  
        return;  
    sysout(trav.data);  
    fDisplay(trav.next);  
}
```

Non tail (head) Recursion

- recursive call is not last statement of recursive function

```
mid rDisplay(Node trav) {  
    if (trav == null)  
        return;  
    rDisplay(trav.next);  
    sysout(trav.data);  
}
```

- depending on number of recursive calls in recursive function, we have following types for direct recursion

Linear recursion

- only one recursive call will be done in a function

```
func() {
```

```
    func()
```

```
}
```

$T(n) = \text{linear}$

Multiple (Tree) recursion

- more than one recursive calls will be done in a function.

```
func() {
```

```
    func() + func()
```

```
}
```

$T(n) = \text{exponential}$

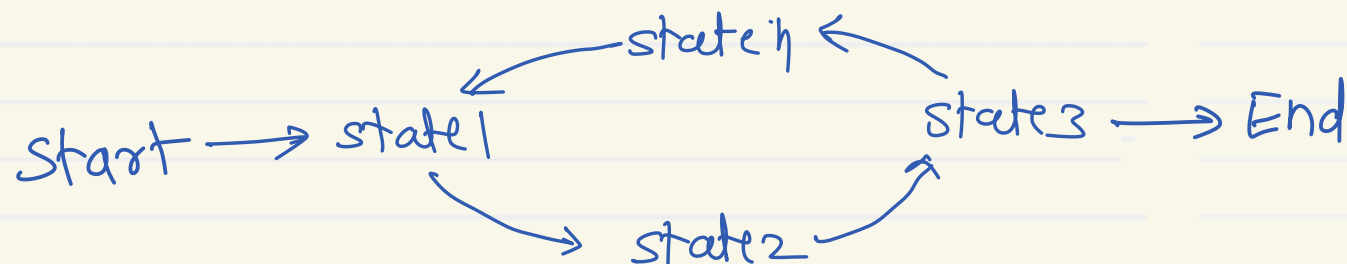
- functions are called indirectly in a cyclic way.

e.g.

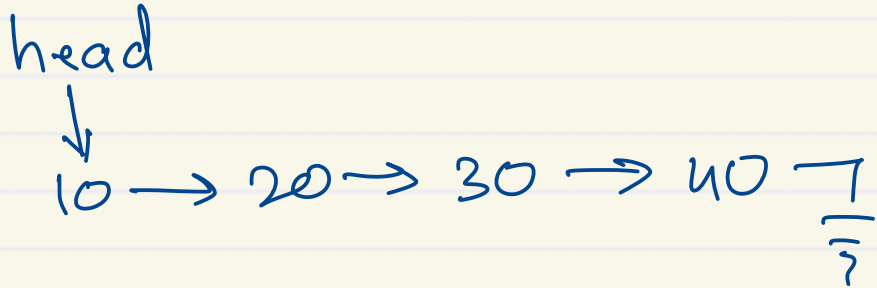
```
fun1() {  
    fun2();  
}  
fun2() {  
    fun1();  
}
```

- indirect recursion is used very rarely.

e.g. multiplayer games
state machines



- Display singly linked list in reverse order. (using loops)



Count no. of node (using loop) = n

while ($n \geq 1$) {

 find n^{th} node of linked list (loop)

 display n^{th} node

$n--$;

}

space : no extra space
- space is constant

$$S(n) = O(1)$$

Time :

$$\text{itrs} = n$$

$$+ n-1$$

$$+ n-2$$

$$+ \dots$$

$$+ 1$$

$$\text{itrs} = n + (n-1) + (n-2) + \dots + 1$$

$$= \frac{n(n+1)}{2}$$

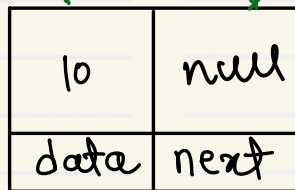
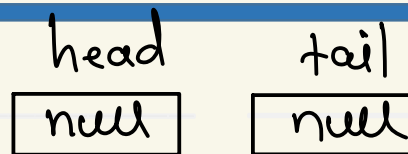
$$T \propto \frac{1}{2}(n^2 + n)$$

$$T(n) = O(n^2)$$

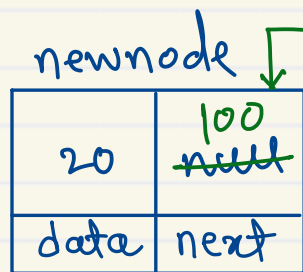
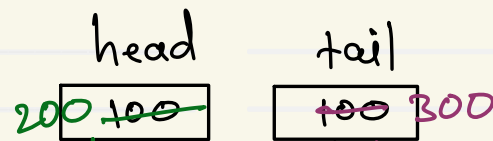
Singly linear linked list with head and tail

```
void addFirst(int value) {
    Node newNode = new Node(value);
    if (head == null)
        head = tail = newNode;
    else {
        newNode.next = head;
        head = newNode;
    }
}
```

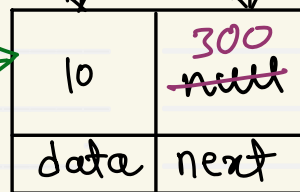
$T(n) = O(1)$



100



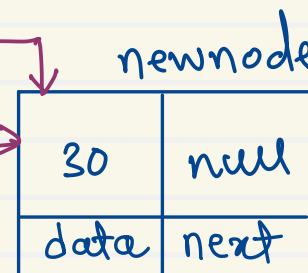
200



100

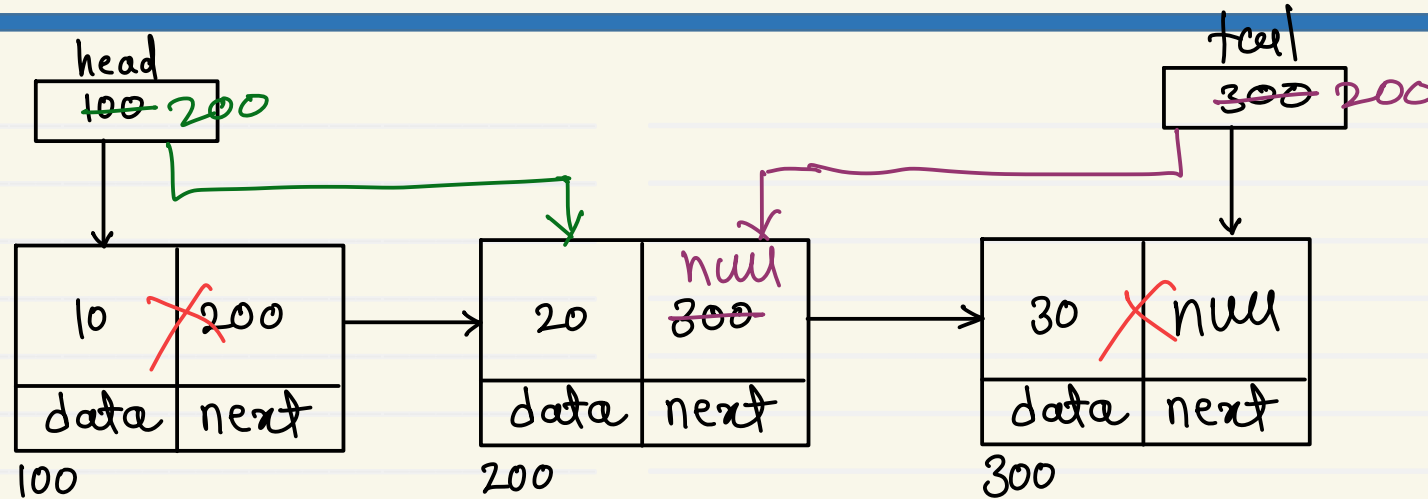
```
void addLast(int value) {
    Node newNode = new Node(value);
    if (head == null)
        head = tail = newNode;
    else {
        tail.next = newNode;
        tail = newNode;
    }
}
```

$T(n) = O(1)$



300

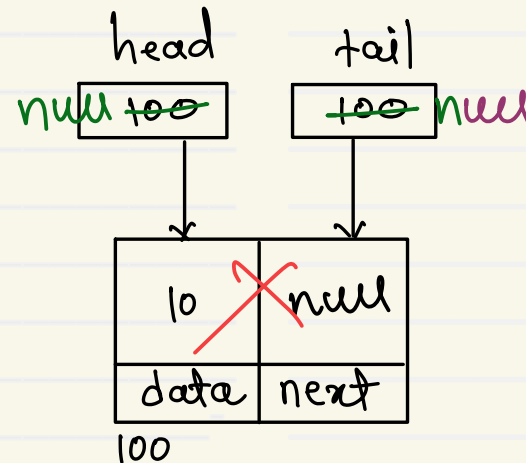
Singly linear linked list with head and tail



```
void deleteFirst ( ) {
    if (head == null)
        return;
    else if (head == tail)
        head = tail = null;
    else
        head = head.next;
```

3

$T(n) = O(1)$

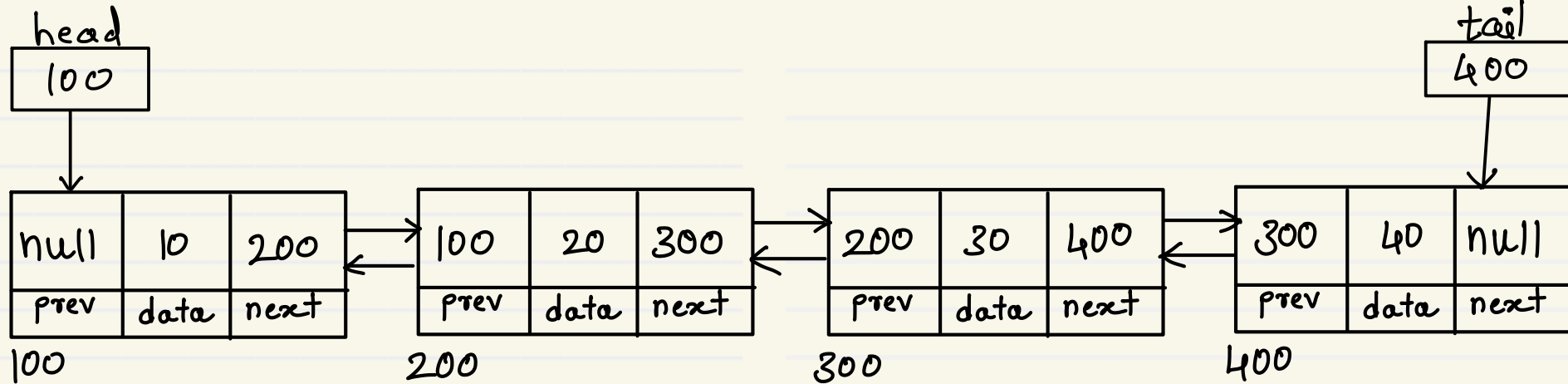


```
void deleteLast ( ) {
    if (head == null)
        return;
    else if (head == tail)
        head = tail = null;
    else {
        Node trav = head;
        while (trav.next.next != null)
            trav = trav.next;
        tail = trav;
        tail.next = null;
```

3

$T(n) = O(n)$

Doubly Linear Linked List - Display



Forward display

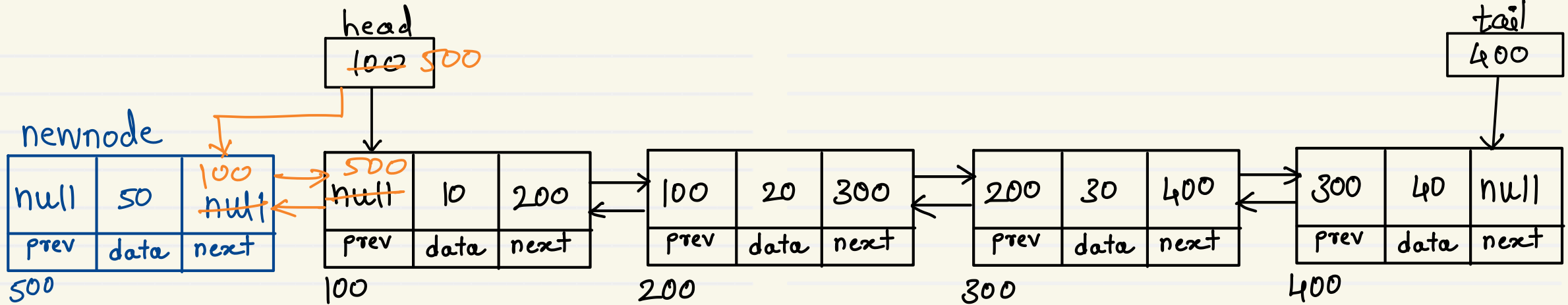
1. Create trav & start on first node
2. visit/print current node
3. go on next node
4. repeat step 2&3 till last node

Backward display

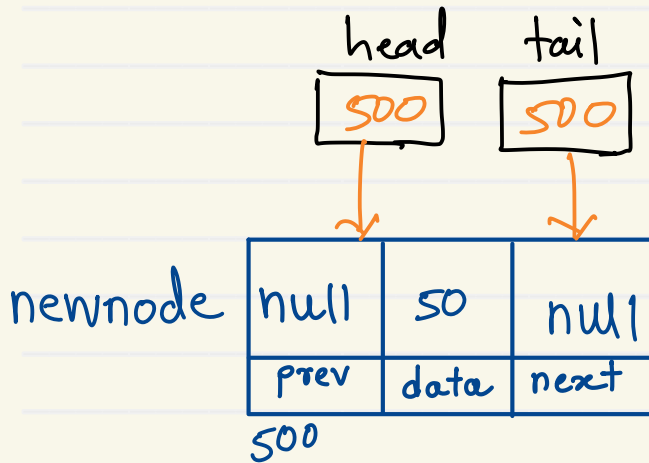
1. Create trav & start on last node
2. visit/print current node
3. go on prev node
4. repeat step 2&3 till First node

$$T(n) = O(n)$$

Doubly Linear Linked List - Add first

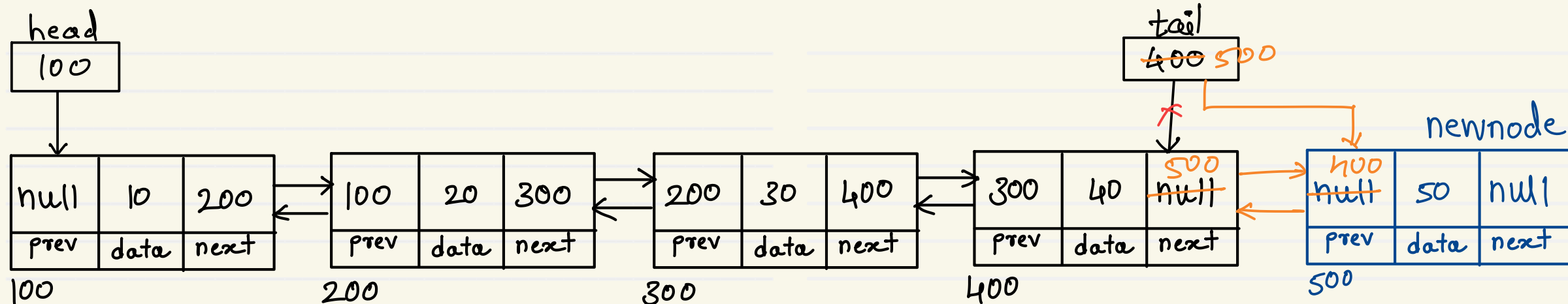


1. Create a newnode with given data
2. if list is empty,
add newnode into head & tail
3. if list is not empty,
 - a. add first node into next of newnode
 - b. add newnode into prev of first node
 - c. move head on newnode

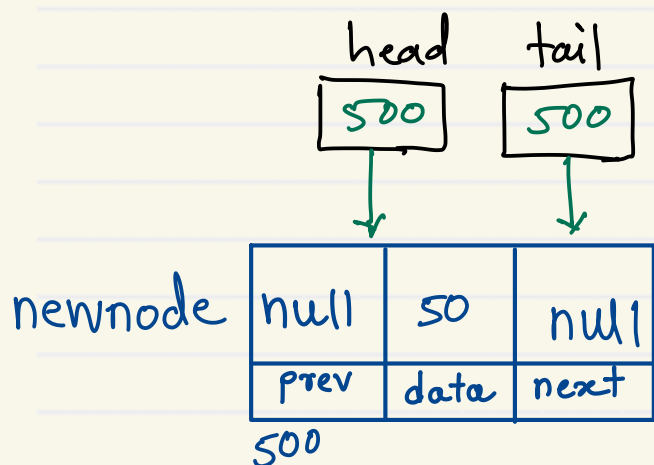


$$T(n) = O(1)$$

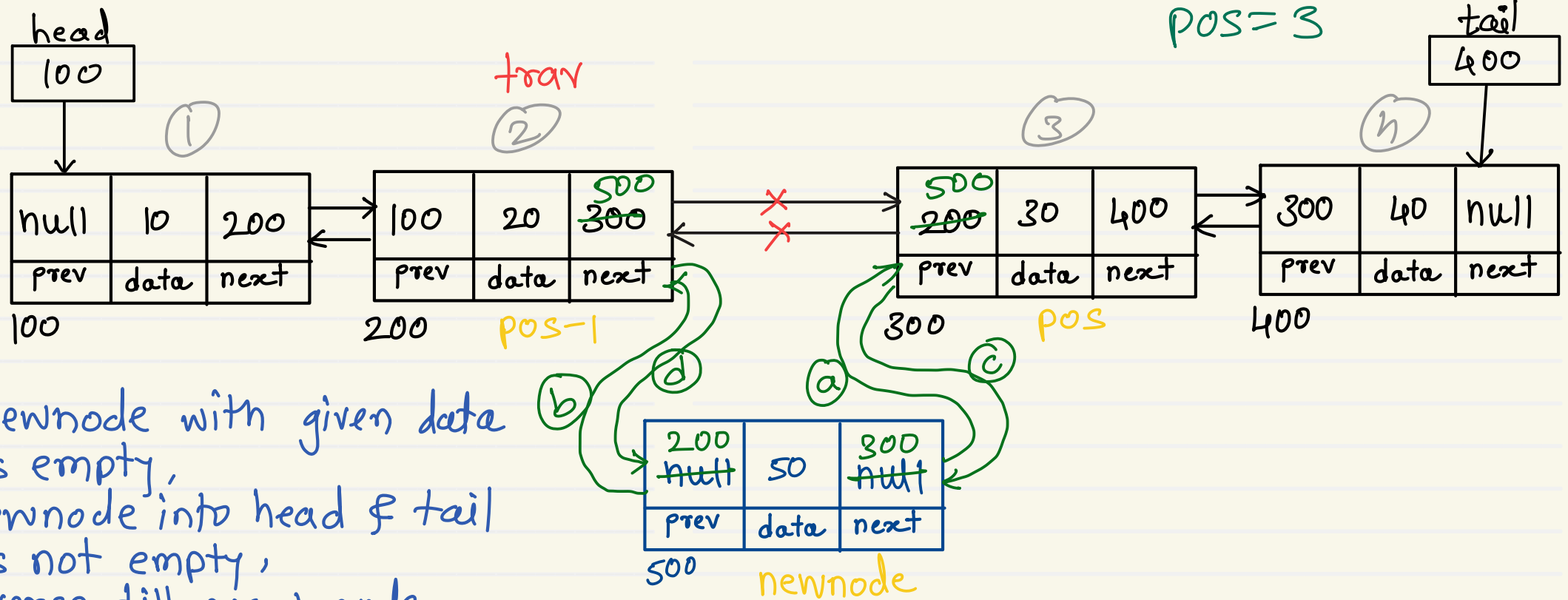
Doubly Linear Linked List - Add last



1. Create a newnode with given data
2. if list is empty,
add newnode into head & tail
3. if list is not empty,
 - a. add last node into prev of newnode
 - b. add newnode into next of last node
 - c. move tail on newnode



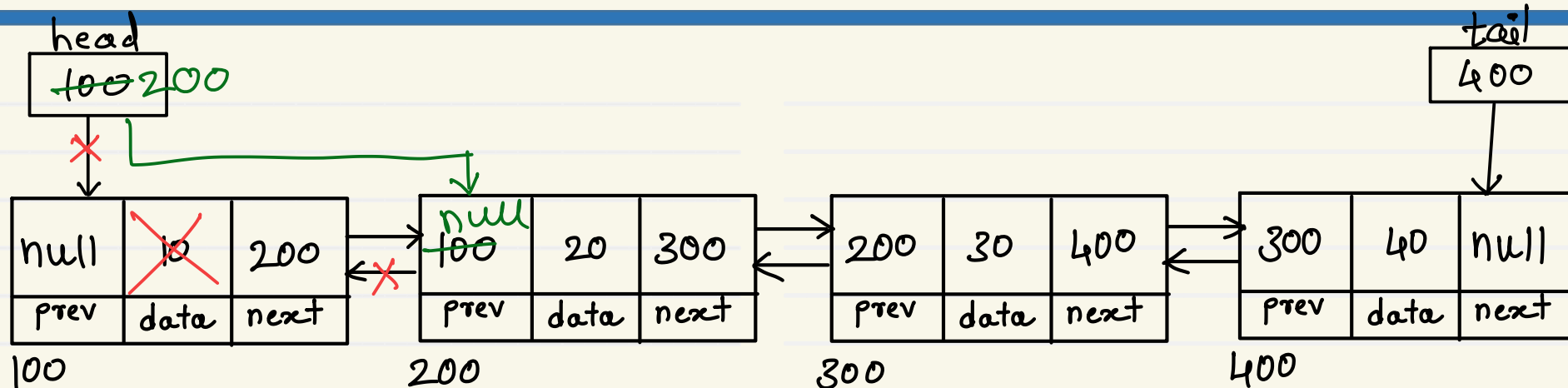
$$T(n) = O(1)$$



1. create newnode with given data
2. if list is empty, add newnode into head & tail
3. if list is not empty,
 - a. traverse till pos-1 node
 - b. add pos node into next of newnode
 - c. add pos-1 node into prev of newnode
 - d. add newnode into prev of pos node
 - e. add newnode into next of pos-1 node

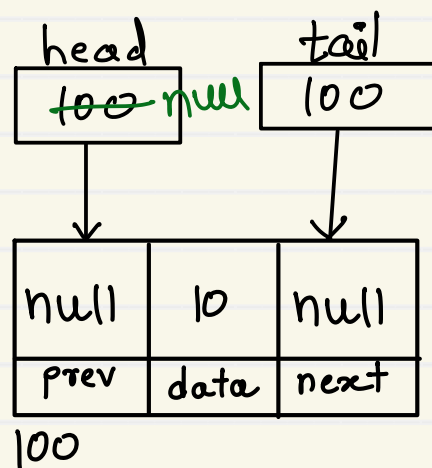
$$T(n) = O(n)$$

Doubly Linear Linked List - Delete first



head = head.next
head.prev = null

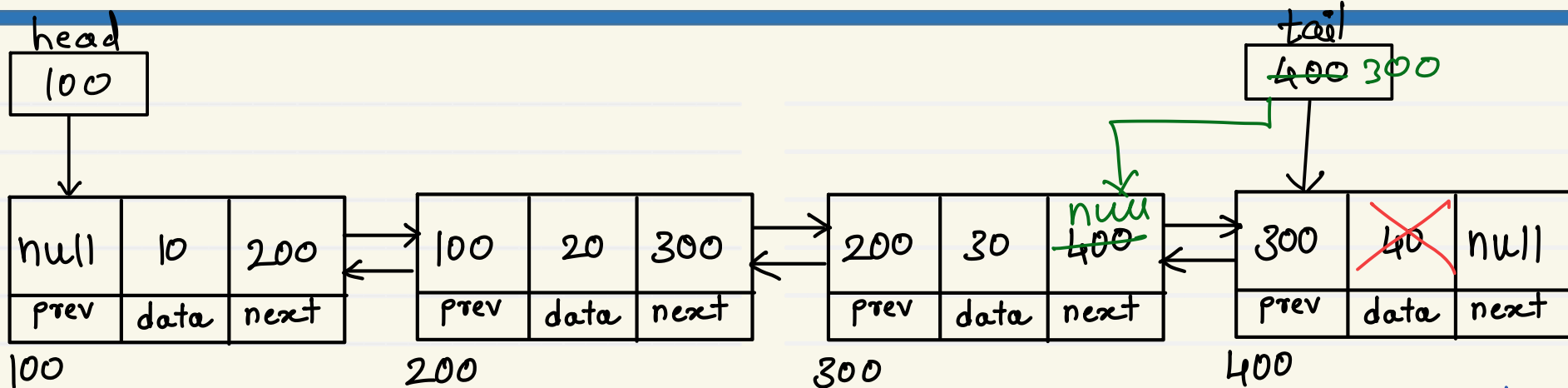
1. if list is empty, return
2. if list has single node,
head = tail = null
3. if list has multiple nodes
 - a. move head on second node
 - b. add null into prev. of second node



head = head.next
head.prev = null
null pointer exception

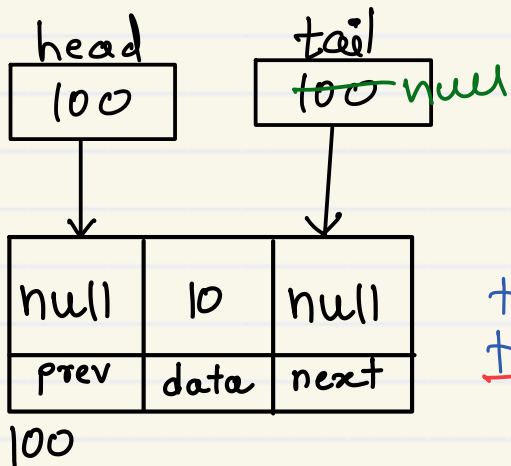
$$T(n) = O(1)$$

Doubly Linear Linked List - Delete last



tail = tail.prev
tail.next = null

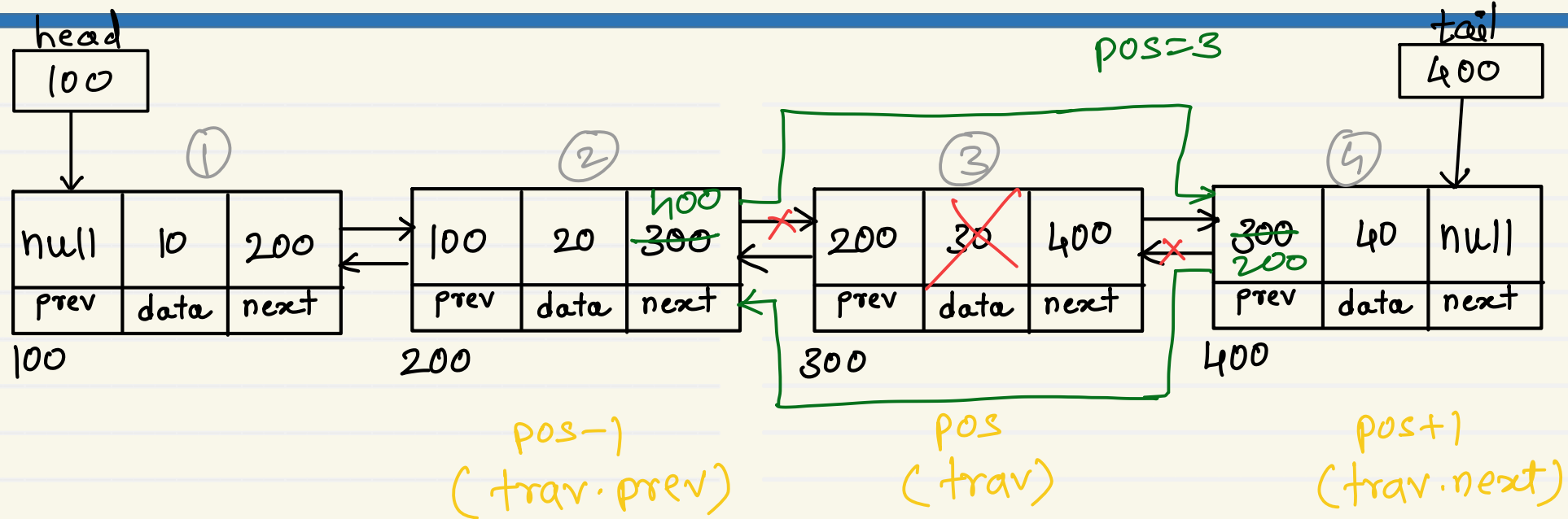
1. if list is empty, return
2. if list has single node,
head = tail = null
3. if list has multiple nodes
 - a. move tail on second last node
 - b. add null into next of second last node



tail = tail.prev
tail.next = null
↳ null pointer exception

$$T(n) = O(1)$$

Doubly Linear Linked List - Delete Position



1. if list is empty, return
2. if list is not empty,
 - a. traverse till pos node
 - b. add pos-1 node into prev of pos+1 node
 - c. add pos+1 node into next of pos-1 node

$$T(n) = O(n)$$